

Exercício Prático: Análise de Desempenho de Árvores de Busca

Objetivo:

Este exercício tem como objetivo comparar, experimentalmente, o desempenho de três estruturas de dados baseadas em árvores: a Árvore Binária de Pesquisa (BST), a Árvore AVL e a Árvore Rubro-Negra (Red-Black Tree). A comparação será focada no custo de construção (tempo de inserção) e na qualidade estrutural resultante (nível médio dos nós) ao processar grandes volumes de dados não ordenados.

Introdução Teórica:

Uma Árvore Binária de Pesquisa (BST) oferece, em média, complexidade $O(\log n)$ para operações de busca, inserção e remoção. No entanto, no pior caso (quando os dados são inseridos de forma ordenada ou quase ordenada), a árvore pode degenerar para uma estrutura semelhante a uma lista ligada, elevando a complexidade para $O(n)$.

Para resolver esse problema, foram criadas as árvores autobalanceadas, como a AVL e a Rubro-Negra. Elas realizam operações extras (rotações) durante as inserções e remoções para garantir que a altura da árvore permaneça logarítmica, mantendo o desempenho delas, mesmo no pior caso.

Este "seguro" de desempenho tem um custo: as operações de inserção em árvores AVL e Rubro-Negra são mais lentas que em uma BST (para o caso médio) devido à sobrecarga do algoritmo de balanceamento.

Neste experimento, vamos medir esse trade-off:

1. **Tempo de Construção:** Qual o custo em tempo para inserir 1 milhão de elementos em cada tipo de árvore?
2. **Qualidade da Estrutura:** Qual a "espalhabilidade" dos nós na árvore resultante? Mediremos isso através do **nível médio dos nós**. Um nível médio baixo indica uma árvore mais compacta e balanceada, resultando em buscas mais rápidas, em média.

Parte 1: Preparação do Ambiente e dos Dados

1.1. Geração dos Arquivos de Teste:

Foram gerados 5 conjuntos de dados, contendo 1 milhão de valores gerados aleatoriamente, a partir do seguinte programa:

Claro! Aqui está um programa em Java que cria um arquivo `teste01.txt` e grava nele uma sequência de 1 milhão de números inteiros gerados aleatoriamente.

Para otimizar o desempenho da gravação, o código utiliza um `BufferedWriter`. Ele armazena os dados em um buffer (uma área de memória temporária) e os grava no disco em blocos maiores, o que é muito mais eficiente do que gravar cada número individualmente.

Código Java: GeradorDeNumeros.java

Java

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Random;

public class GeradorDeNumeros {

    public static void main(String[] args) {

        // Define o nome do arquivo e a quantidade de números a serem gerados.
        String nomeArquivo = "teste01.txt";
        int quantidadeDeNumeros = 1_000_000; // O underscore melhora a legibilidade de números grandes.

        // Instancia um gerador de números aleatórios.
        Random random = new Random();

        // O bloco "try-with-resources" garante que o BufferedWriter será fechado automaticamente
        // ao final do bloco, mesmo que ocorra um erro.
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(nomeArquivo))) {

            System.out.println("Iniciando a gravação de " + quantidadeDeNumeros + " números no
arquivo " + nomeArquivo + "...");

            // Loop para gerar e gravar cada número.
            for (int i = 0; i < quantidadeDeNumeros; i++) {
                // Gera um número inteiro aleatório.
                int numeroAleatorio = random.nextInt();

                // Converte o número para String e o escreve no arquivo.
                writer.write(Integer.toString(numeroAleatorio));

                // Adiciona uma quebra de linha para que cada número fique em sua própria linha.
                writer.newLine();
            }

            System.out.println("Gravação concluída com sucesso!");

        } catch (IOException e) {
            // Captura e informa possíveis erros de I/O (Input/Output).
            System.err.println("Ocorreu um erro ao escrever no arquivo: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

}

É fundamental que os mesmos 5 arquivos sejam usados para testar as 3 implementações de árvore, garantindo que a comparação seja justa.

1.2. Implementação das Estruturas de Dados:

Você deve ter implementações funcionais das seguintes árvores. Cada árvore deve conter, no mínimo, um método `insert(int valor)`.

1. **Árvore Binária de Pesquisa (BST)**
2. **Árvore AVL**
3. **Árvore Rubro-Negra (Red-Black Tree)**

Parte 2: Execução dos Experimentos

2.1. Criação do Programa de Teste (Test Harness):

Crie um programa principal em Java que irá orquestrar os testes. O fluxo de execução para cada um dos 5 arquivos de teste (aleatórios) e depois para os dois arquivos de teste (crescente e decrescente) deve ser o seguinte:

Para cada tipo de árvore (BST, AVL, Rubro-Negra):

1. Instancie uma árvore vazia do tipo correspondente.
2. Leia o arquivo de teste (ex: `teste01.txt`) linha por linha. Para cada número lido, armazene-o em uma estrutura de dados conveniente.
3. Registre o tempo de início do sistema: `long startTime = System.nanoTime();`.
4. Insira um a um os números obtidos no passo 2.
5. Após a inserção do último número, registre o tempo final: `long endTime = System.nanoTime();`.
6. Calcule o tempo total de execução em milissegundos: `long duration = (endTime - startTime) / 1_000_000;`.
7. Calcule o nível médio dos nós da árvore recém-criada (ver seção 2.2).
8. Imprima os resultados para esta execução, por exemplo: "Tipo: BST, Arquivo: `teste01.txt`, Tempo de Inserção: XXXX ms, Nível Médio: XX.XX".
9. Limpe a árvore ou crie uma nova instância para o próximo teste.

2.2. Cálculo do Nível Médio dos Nós:

O nível médio é a soma do nível de cada nó na árvore, dividida pelo número total de nós. Para calculá-lo, você precisará implementar um método que percorra a árvore.

- O nível da raiz é 0.
- O nível de qualquer outro nó é `nível do pai + 1`.

Você pode criar uma função recursiva auxiliar que recebe um nó e seu nível como parâmetros.

Exemplo de implementação em Java (a ser adaptado para a sua classe de Nó):

Java

```
// Variável de instância na sua classe de árvore para armazenar a soma
private long somaDosNiveis = 0;

// Método público que inicia o cálculo
public double calcularNivelMedio() {
    if (raiz == null) {
        return 0;
    }
    somaDosNiveis = 0;
    // O número total de nós é 1.000.000 neste exercício
    int totalDeNos = 1_000_000;

    // Inicia o percurso recursivo a partir da raiz, que está no nível 0
    calcularSomaNiveis(this.raiz, 0);

    return (double) somaDosNiveis / totalDeNos;
}

// Método recursivo privado
private void calcularSomaNiveis(No no, int nivel) {
    if (no == null) {
        return;
    }

    // Adiciona o nível do nó atual à soma total
    this.somaDosNiveis += nivel;

    // Continua o percurso para os filhos, incrementando o nível
    calcularSomaNiveis(no.getEsquerda(), nivel + 1);
    calcularSomaNiveis(no.getDireita(), nivel + 1);
}
```

Parte 3: Relatório de Análise

Após coletar os dados de tempo de execução e nível médio para as 3 árvores nos 5 arquivos de teste, organize-os e prepare um relatório com a seguinte estrutura:

Modelo de Relatório de Análise de Desempenho

1. Capa:

- Nome da Disciplina
- Título do Exercício
- Nome do(s) Aluno(s)
- Data

2. Introdução:

- Breve descrição do problema: a necessidade de estruturas de dados eficientes para busca e o problema da degeneração da BST.
- Apresentação das estruturas de dados analisadas (BST, AVL, Rubro-Negra).
- Objetivo do experimento: comparar o tempo de construção e a qualidade estrutural (nível médio dos nós) das três árvores ao inserir 1 milhão de inteiros aleatórios.

3. Metodologia:

- **Ambiente de Teste:** Descreva o hardware (Processador, Memória RAM) e software (Sistema Operacional, Versão do Java/JDK) utilizados.
- **Geração de Dados:** Explique como os 7 arquivos de teste foram gerados.
- **Medição de Desempenho:** Explique como o tempo de execução foi medido (`System.nanoTime()`) e como o nível médio dos nós foi calculado.

4. Resultados:

- Apresente os dados coletados de forma clara em uma tabela. Calcule também as médias de tempo e de nível para cada tipo de árvore, considerando os 5 testes com dados gerados aleatoriamente e após, com os 2 testes com dados ordenados.

Tipo de Árvore	Arquivo de Teste	Tempo de Inserção (ms)	Nível Médio dos Nós
BST	teste01.txt	...	...
BST	teste02.txt	...	...
...	...	...	...

BST	MÉDIA	...	...
AVL	teste01.txt	...	...
...	...	...	...
AVL	MÉDIA	...	...
Rubro-Negra	teste01.txt	...	...
...	...	...	...
Rubro-Negra	MÉDIA	...	...

5. Análise e Discussão:

- **Análise de Tempo:** Qual algoritmo teve o menor tempo médio de construção? Por que você acha que isso aconteceu? Relacione com a sobrecarga dos algoritmos de balanceamento.
- **Análise da Estrutura:** Qual algoritmo produziu árvores com o menor nível médio de nós? O que esse resultado indica sobre a "compacidade" e o balanceamento da árvore?
- **Trade-off (Custo vs. Benefício):** Discuta a relação entre o tempo de inserção e o nível médio. Uma inserção mais lenta (AVL/Rubro-Negra) resultou em uma árvore melhor para futuras buscas? Justifique.
- **Importância dos Múltiplos Testes:** Por que foi importante usar 5 conjuntos de dados aleatórios em vez de apenas um? Os resultados variaram muito entre os arquivos? Por que foi importante usar mais 2 conjuntos de dados ordenados (crescente e decrescente)? Como os resultados desses dois conjuntos foram semelhantes ou diferentes dos conjuntos aleatórios? Para os dados ordenados o tempo de inserção e a qualidade da árvore foram discrepantes dos dados gerados aleatoriamente?

6. Conclusão:

- Resuma os resultados do experimento. Qual árvore se mostrou mais eficiente para a tarefa de inserção em massa de dados aleatórios, considerando tanto o tempo quanto a qualidade da estrutura gerada?
- Com base na sua análise, em que tipo de aplicação você recomendaria o uso de uma BST simples em vez de uma árvore autobalanceada?

7. Anexo (Código-Fonte):

- Anexe o código-fonte completo das suas implementações das árvores e do programa de teste.